

Seguridad en Sistemas Operativos

Control de acceso, usuarios, permisos y auditoría en entorno Linux

Materia:	Arquitectura y Sistemas Operativos (AYSO)
Alumno:	Agustín Fernández – agustin.fernandez.official@gmail.com
Profesor/Tutor:	Sebastián Sáez
Fecha de entrega:	25/06/2026

1. Introducción

La seguridad informática constituye uno de los pilares fundamentales de la administración de sistemas operativos modernos. En un contexto donde la información se ha convertido en uno de los activos más valiosos tanto para organizaciones como para usuarios individuales, resulta indispensable comprender y aplicar correctamente los mecanismos de control de acceso que ofrecen los sistemas operativos tipo Unix/Linux.

Se eligió este tema por su relevancia dentro de la formación como técnico en programación: la gestión de usuarios, permisos y auditoría son tareas habituales en la administración de cualquier sistema, y su comprensión resulta clave tanto para el desarrollo de software seguro como para la correcta configuración y mantenimiento de los entornos sobre los que dicho software se ejecuta.

El objetivo principal de este trabajo es demostrar, mediante un caso práctico desarrollado en un entorno virtualizado con Debian 12, cómo los mecanismos nativos de Linux —permisos de archivos, gestión de usuarios y grupos, auditoría de eventos y delegación de privilegios mediante sudo— permiten proteger eficazmente la confidencialidad de la información sensible sin requerir herramientas externas.

2. Marco Teórico

Los sistemas operativos tipo Unix/Linux incorporan, desde su diseño original, un modelo de seguridad multiusuario que permite controlar de manera granular qué procesos y qué personas pueden acceder a qué recursos. Este modelo se sostiene sobre varios pilares conceptuales que se describen a continuación.

Permisos de archivos

Cada archivo y directorio en Linux posee un conjunto de permisos asociados a tres categorías de sujetos: el propietario (owner), el grupo (group) y el resto de los usuarios del sistema (others). Para cada categoría se definen tres permisos básicos: lectura (read, r), escritura (write, w) y ejecución (execute, x). Estos permisos se representan habitualmente en notación simbólica (por ejemplo, rw-----) o en notación octal (por ejemplo, 600), donde cada cifra octal codifica los tres permisos de una categoría. El comando `chmod` permite modificar estos

permisos, mientras que `chown` permite cambiar el propietario y el grupo asociado a un archivo.

Usuarios y grupos

Linux es un sistema multiusuario en el que cada persona que accede al sistema posee una cuenta identificada por un nombre de usuario y un UID (User ID) numérico único. Los usuarios pueden agruparse en grupos (con su correspondiente GID), lo cual permite asignar permisos compartidos a conjuntos de usuarios sin tener que otorgarlos de forma individual. La creación de usuarios se realiza habitualmente mediante el comando `adduser` (o `useradd`), que crea automáticamente el directorio personal del usuario y su grupo primario.

Control de acceso basado en roles (RBAC)

El modelo de Control de Acceso Basado en Roles propone asignar permisos no directamente a usuarios individuales, sino a roles que representan funciones dentro de una organización (por ejemplo, administrador, operador, auditor), asociando luego a cada usuario uno o más roles. Aunque Linux no implementa RBAC de forma nativa en su modelo clásico de permisos, el mecanismo de `sudo` permite aproximar este concepto al delegar de forma controlada y documentada qué comandos específicos puede ejecutar cada usuario, actuando como una forma simplificada de control de acceso basado en funciones.

Auditoría de eventos con auditd

El subsistema de auditoría del kernel de Linux, gestionado por el demonio `auditd`, permite registrar de forma detallada eventos de seguridad relevantes: accesos a archivos específicos, ejecución de comandos privilegiados, cambios de configuración, entre otros. Mediante el comando `auditctl` es posible definir reglas de vigilancia (`watches`) sobre archivos o directorios concretos, especificando qué tipo de acceso debe registrarse (lectura, escritura, ejecución, cambio de atributos). Los eventos registrados pueden consultarse posteriormente con herramientas como `ausearch` o `aureport`, lo cual resulta fundamental para la trazabilidad y la respuesta ante incidentes de seguridad.

Delegación de privilegios mediante sudo

El comando sudo (superuser do) permite que usuarios no privilegiados ejecuten comandos específicos con permisos elevados, sin necesidad de conocer la contraseña del superusuario ni de iniciar sesión como root. La configuración de sudo se gestiona mediante el archivo `/etc/sudoers` (editado de forma segura con `visudo`) o mediante archivos individuales dentro del directorio `/etc/sudoers.d/`. Este mecanismo permite aplicar el principio de privilegio mínimo (least privilege), otorgando a cada usuario únicamente los permisos estrictamente necesarios para cumplir su función, y no más.

3. Caso Práctico

El caso práctico desarrollado consiste en la implementación de un escenario de control de acceso a un archivo confidencial dentro de una máquina virtual con Debian 12, utilizando los conceptos descritos en el marco teórico. El problema a resolver fue el siguiente: proteger un archivo con información sensible de modo que únicamente un usuario autorizado pueda acceder a él, registrar todos los intentos de acceso (exitosos y fallidos) mediante un sistema de auditoría, y delegar de forma controlada un permiso puntual a un usuario sin privilegios.

3.1. Creación de usuarios

Se crearon dos usuarios con distinto nivel de acceso sobre el sistema:

```
sudo adduser autorizado
sudo adduser noautorizado
```

```

linuxdebian@MaquinaLinux:~$ sudo adduser noautorizado
Añadiendo el usuario 'noautorizado' ...
Añadiendo el nuevo grupo 'noautorizado' (1001) ...
Adding new user 'noautorizado' (1001) with group 'noautorizado (1001)' ...
Creando el directorio personal '/home/noautorizado' ...
Copiando los ficheros desde '/etc/skel' ...
Nueva contraseña:
Vuelva a escribir la nueva contraseña:
passwd: contraseña actualizada correctamente
Cambiando la información de usuario para noautorizado
Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado
Nombre completo []:
Número de habitación []:
Teléfono del trabajo []:
Teléfono de casa []:
Otro []:
¿Es correcta la información? [S/n] s
Adding new user 'noautorizado' to supplemental / extra groups 'users' ...
Añadiendo al usuario 'noautorizado' al grupo 'users' ...
-----
linuxdebian@MaquinaLinux:~$ sudo adduser autorizado
Añadiendo el usuario 'autorizado' ...
Añadiendo el nuevo grupo 'autorizado' (1002) ...
Adding new user 'autorizado' (1002) with group 'autorizado (1002)' ...
Creando el directorio personal '/home/autorizado' ...
Copiando los ficheros desde '/etc/skel' ...
Nueva contraseña:
Vuelva a escribir la nueva contraseña:
passwd: contraseña actualizada correctamente
Cambiando la información de usuario para autorizado
Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado
Nombre completo []:
Número de habitación []:
Teléfono del trabajo []:
Teléfono de casa []:
Otro []:
¿Es correcta la información? [S/n] s
Adding new user 'autorizado' to supplemental / extra groups 'users' ...
Añadiendo al usuario 'autorizado' al grupo 'users' ...
linuxdebian@MaquinaLinux:~$ cat /etc/passwd | grep -E "autorizado|noautorizado"
bash: grep: orden no encontrada
linuxdebian@MaquinaLinux:~$ cat /etc/passwd | grep -E "autorizado|noautorizado"
noautorizado:x:1001:1001:,,,:/home/noautorizado:/bin/bash
autorizado:x:1002:1002:,,,:/home/autorizado:/bin/bash
linuxdebian@MaquinaLinux:~$ █

```

3.2. Restricción de acceso mediante permisos

Se creó un archivo confidencial dentro de un directorio dedicado, asignándole como propietario al usuario autorizado y restringiendo el acceso exclusivamente a este mediante el permiso octal 600 (lectura y escritura solo para el propietario):

```

sudo mkdir /seguro
sudo touch /seguro/datos_confidenciales.txt
sudo chown autorizado:autorizado /seguro/datos_confidenciales.txt
sudo chmod 600 /seguro/datos_confidenciales.txt
ls -l /seguro/

```

```

-----
linuxdebian@MaquinaLinux:~$ sudo chown autorizado:autorizado /seguro/datos_confidenciales.txt
linuxdebian@MaquinaLinux:~$ sudo chmod 600 /seguro/datos_confidenciales.txt
linuxdebian@MaquinaLinux:~$ ls -l /seguro/
total 0
-rw----- 1 autorizado autorizado 0 jun 21 14:54 datos_confidenciales.txt
linuxdebian@MaquinaLinux:~$ █

```

3.3. Validación de acceso denegado

Se verificó que el usuario noautorizado, al no poseer permisos sobre el archivo, no puede acceder a su contenido:

```
su - noautorizado
cat /seguro/datos_confidenciales.txt
```

```
linuxdebian@MaquinaLinux:~$ su - noautorizado
Contraseña:
noautorizado@MaquinaLinux:~$ cat /seguro/datos_confidenciales.txt
cat: /seguro/datos_confidenciales.txt: Permiso denegado
noautorizado@MaquinaLinux:~$
```

3.4. Validación de acceso autorizado

Se verificó que el usuario autorizado, propietario del archivo, puede leer y escribir su contenido sin restricciones:

```
su - autorizado
cat /seguro/datos_confidenciales.txt
echo "informacion confidencial de prueba" >> /seguro/datos_confidenciales.txt
cat /seguro/datos_confidenciales.txt
```

```
linuxdebian@MaquinaLinux:~$ su - autorizado
Contraseña:
autorizado@MaquinaLinux:~$ cat /seguro/datos_confidenciales.txt
autorizado@MaquinaLinux:~$ echo "informacion confidencial de prueba" >> /seguro/datos_confidenciales.txt
autorizado@MaquinaLinux:~$ cat /seguro/datos_confidenciales.txt
informacion confidencial de prueba
autorizado@MaquinaLinux:~$
```

3.5. Instalación y configuración de auditd

Se instaló el sistema de auditoría auditd junto con sus complementos, se activó el servicio y se configuró una regla de vigilancia sobre el archivo confidencial:

```
sudo apt install auditd audispd-plugins
sudo systemctl enable auditd --now
sudo auditctl -w /seguro/datos_confidenciales.txt -p rwx -k acceso_datos
sudo auditctl -l
```


otorgarle ningún otro privilegio sobre el sistema. La regla se incorporó en un archivo independiente dentro de `/etc/sudoers.d/`, validándose su sintaxis antes de aplicarla:

```
echo 'noautorizado ALL=(ALL) NOPASSWD: /bin/cat
/seguro/datos_confidenciales.txt' \
| sudo tee /etc/sudoers.d/noautorizado_cat
sudo chmod 440 /etc/sudoers.d/noautorizado_cat
sudo visudo -c
```

```
linuxdebian@MaquinaLinux:~$ sudo visudo -c
/etc/sudoers: parsed OK
/etc/sudoers.d/README: parsed OK
/etc/sudoers.d/noautorizado_cat: parsed OK
linuxdebian@MaquinaLinux:~$
```

Finalmente, se comprobó que el privilegio otorgado está correctamente acotado: el usuario puede ejecutar `cat` únicamente sobre el archivo autorizado, mientras que cualquier otro intento de uso de `sudo` —como el acceso a `/etc/shadow`— es rechazado explícitamente por el sistema:

```
su - noautorizado
sudo cat /seguro/datos_confidenciales.txt    # éxito
sudo cat /etc/shadow                        # denegado
```

```
noautorizado@MaquinaLinux:~$ sudo cat /seguro/datos_confidenciales.txt
informacion confidencial de prueba
noautorizado@MaquinaLinux:~$ sudo cat /etc/shadow
[sudo] contraseña para noautorizado:
Sorry, user noautorizado is not allowed to execute '/usr/bin/cat /etc/shadow' as root o
n MaquinaLinux.DominioLinux.
noautorizado@MaquinaLinux:~$
```

4. Metodología Utilizada

- Investigación previa de los conceptos de permisos, usuarios, auditoría y `sudo` en documentación oficial de Debian y manuales del sistema (man pages).
- Configuración de un entorno virtualizado con Oracle VM VirtualBox y Debian 12 (64 bits), con 4096 MB de RAM y 2 procesadores asignados.
- Desarrollo incremental del caso práctico, validando cada paso (creación de usuarios, permisos, auditoría, `sudo`) antes de avanzar al siguiente, mediante capturas de pantalla que evidencian cada resultado.

- Organización del material de evidencia (capturas de pantalla) en un repositorio público de GitHub, con documentación en formato README.

5. Resultados Obtenidos

- Se validó correctamente el bloqueo de acceso a un archivo confidencial para un usuario sin permisos, obteniendo el mensaje de error esperado ("Permiso denegado").
- Se validó el acceso completo (lectura y escritura) por parte del usuario propietario del archivo.
- Se instaló y configuró exitosamente el sistema de auditoría auditd, registrando una regla de vigilancia sobre el archivo confidencial.
- Se registraron en el log de auditoría tanto el intento de acceso fallido como el acceso exitoso, pudiendo identificarse en cada evento el usuario, el comando ejecutado, el resultado (success=no / success=yes) y la marca de tiempo exacta.
- Se configuró y validó una regla de sudo que delega un privilegio mínimo y específico, comprobando que el usuario beneficiario solo puede ejecutar el comando autorizado sobre el recurso autorizado, y que cualquier otro intento de uso de sudo es rechazado.
- Durante el desarrollo se presentaron dificultades menores —errores de tipeo en comandos (chown en lugar de chmod, nombres de paquetes mal escritos, rutas sin espacios) y un error de sintaxis al editar el archivo /etc/sudoers manualmente con un editor de texto— que fueron identificados y corregidos satisfactoriamente, optando finalmente por un método más seguro (archivo independiente en /etc/sudoers.d/ mediante el comando tee) para evitar riesgos de romper la configuración del sistema.
- No se presentaron limitaciones relevantes de hardware ni de software para completar el caso práctico; el principal inconveniente fue el espacio en disco inicialmente disponible en la máquina virtual, que se resolvió liberando archivos de caché y kernels antiguos sin necesidad de redimensionar el disco virtual.

6. Conclusiones

El desarrollo de este trabajo permitió comprender en profundidad, mediante la práctica directa, cómo los mecanismos nativos de seguridad de los sistemas operativos Linux — permisos de archivos, gestión de usuarios, auditoría de eventos y delegación de privilegios—

se combinan para construir una política de seguridad efectiva sobre información sensible, sin requerir herramientas externas al propio sistema operativo.

También pudimos constatar que el sistema de auditoría no solo registra los accesos no autorizados, sino también los autorizados, lo cual resulta fundamental para la trazabilidad completa de quién accede a qué recursos y cuándo, un aspecto clave en cualquier entorno profesional de administración de sistemas.

Asimismo, la implementación del principio de privilegio mínimo mediante sudo permitió comprender de forma concreta una de las máximas más importantes de la seguridad informática: otorgar a cada usuario únicamente el acceso estrictamente necesario para cumplir su función, ni más ni menos. Este enfoque resulta aplicable a cualquier entorno de administración de sistemas, independientemente del rol específico que se desempeñe dentro de una organización.

7. Bibliografía

- Debian Project. (s.f.). Documentación oficial de Debian. Recuperado el 25/06/2026 de <https://www.debian.org/doc/>
- Arch Linux Community. (s.f.). Security — ArchWiki. Recuperado el 25/06/2026 de <https://wiki.archlinux.org/title/Security>
- The Linux man-pages project. (s.f.). auditd(8) — Linux manual page. Recuperado el 25/06/2026 de <https://man7.org/linux/man-pages/man8/auditd.8.html>
- The Linux man-pages project. (s.f.). sudoers(5) — Linux manual page. Recuperado el 25/06/2026 de <https://man7.org/linux/man-pages/man5/sudoers.5.html>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10ª ed.). John Wiley & Sons. Capítulo 17, "Protection".

8. Anexos

El repositorio del proyecto:

<https://github.com/Agustin-E-Fernandez/TP-Integrador-Seguridad-AYSO>